

MPTCP vs MP-QUIC

A Comparative Study of Multipath Transport: Mechanisms, Measurement, and Evaluation

Kaushal Rathi (24110290) Krina Paghadar (24110169)
Pratik Kumar Dhakar (24110277) Swamini More (24110211)

CS 331 - Computer Networks

Tagging key used throughout this report: [MEASURED] = our own experimental run; [LIT: n] = drawn from the numbered reference at that position; [PREDICTED] = our own mechanistic reasoning, not independently measured. Every specific number in this report carries one of these three tags.

Abstract

Multipath transport lets a connection use more than one network path at once, improving throughput and tolerating path failure. This report studies two approaches: Multipath TCP (MPTCP), which extends TCP, and Multipath QUIC (MP-QUIC), which extends QUIC. Our own hands-on work covers MPTCP only: we built a Linux-kernel MPTCP testbed and ran four controlled experiments (symmetric paths, asymmetric bandwidth/delay, asymmetric paths with loss, and mid-transfer path failure), measuring throughput, RTT, jitter, path utilisation, and failover behaviour. All four runs matched our pre-stated expectations. Given the two-week project window, MP-QUIC was not independently implemented; its mechanisms, and its comparison against MPTCP, TCP, and single-path QUIC, are built entirely from published research, with our own reasoning used only to project how MP-QUIC’s documented design would behave under our four scenarios, clearly separated from measured data. We also include a scoped, design-level note on the inter-library question of TQUIC versus Cloudflare’s quiche, since no multipath timing data exists for that pair. We close with per-application recommendations (video, file transfer, VoIP) that are our own synthesis of the above, not new findings. The central conclusion, consistent with the literature reviewed, is that there is no universal winner: the better protocol depends on path symmetry, loss, and the application’s tolerance for delay versus jitter.

1. Introduction: Problem Statement and Objectives

Multipath transport protocols exist because end hosts are increasingly multi-homed (Wi-Fi plus LTE, dual-NIC servers, IPv4/IPv6), a scenario RFC 8684 itself lists as a core motivating case [LIT: 1]. A single TCP or QUIC connection is pinned to one path, wasting available capacity and making the connection fragile: if the one path in use fails, the connection fails with it. Multipath transport addresses two goals that are somewhat in tension:

- **Reliability:** survive the failure of any one path by keeping the connection alive on the others.
- **Throughput:** combine the capacity of multiple paths into one logical connection.

MPTCP, standardised in RFC 8684 [LIT: 1], achieves this by extending TCP with additional “subflows” over the same logical connection. MP-QUIC does the same for QUIC, but QUIC multipath is not yet one settled IETF standard; it exists as competing research prototypes.

Objectives of this project:

1. Implement and measure MPTCP end to end under symmetric, asymmetric, lossy, and failure conditions.
2. Understand MP-QUIC’s mechanisms and how they differ from MPTCP’s, using the published literature.
3. Compare TCP, MPTCP, QUIC, and MP-QUIC on throughput, latency, and reliability, tagging every number by its source.
4. Give practical, application-specific recommendations (video streaming, file transfer, VoIP).

Scope statement. This is a two-week project. MPTCP is fully implemented and measured (four experiments, Section 5.1). MP-QUIC is not independently implemented or run; it is scoped to mechanism study plus literature-sourced numbers and our own labelled reasoning. This is a deliberate scope decision, stated plainly rather than implied away.

2. Background: Protocol Mechanisms

2.1 MPTCP: Handshakes and Path Management

MPTCP layers a second protocol state machine on top of ordinary TCP, using TCP options, so a non-MPTCP-aware middlebox simply sees a normal TCP stream [LIT: 1].

- **MP_CAPABLE:** sent in the initial SYN/SYN-ACK/ACK exchange. It confirms both endpoints support MPTCP and exchanges keys used to derive an HMAC-SHA256-based token that authenticates every later subflow as belonging to this connection [LIT: 1].
- **MP_JOIN:** used when adding a new subflow. It carries the connection token, a random nonce, and an HMAC computed from both sides' keys and nonces, proving the joining subflow belongs to the same MPTCP connection [LIT: 1].
- **Address announcement:** two policies exist. **Signal** mode advertises an address via **ADD_ADDR** and lets the peer decide whether to open a subflow to it; **fullmesh** mode proactively opens subflows between every known address pair [LIT: 1].
- **Subflow limits:** parameters such as **add_addr_accepted** and **subflows** cap how many additional paths or addresses a connection will accept [LIT: 1].

2.2 Coupled Congestion Control

Running ordinary single-path congestion control independently on each subflow would let an MPTCP connection take more than its fair share of a bottleneck shared with regular TCP flows. Three coupled algorithms address this, in the order they were proposed:

- **LIA (Linked Increases Algorithm)**, RFC 6356 [LIT: 2]: the original design; couples subflow windows so the connection is no more aggressive on a shared bottleneck than a single TCP flow. Khalili et al. show LIA can still, in some topologies, reduce the throughput of the regular TCP flows it shares a bottleneck with without benefiting the MPTCP flow itself [LIT: 3].
- **OLIA (Opportunistic LIA)** [LIT: 3]: proposed by Khalili et al. after identifying the LIA issue above; redesigns the window-increase rule to fix it while keeping the multipath throughput benefit.
- **BALIA (Balanced Linked Adaptation)** [LIT: 4]: Peng et al. show LIA and OLIA sit at different points on a friendliness-versus-responsiveness trade-off (LIA can be unfriendly to single-path TCP, OLIA can be slow to react when subflow RTTs are similar) and propose BALIA to balance the two properties.

2.3 Scheduling Algorithms

Once congestion control clears packets to send, the scheduler decides which subflow carries each packet.

Scheduler	Behaviour	Best suited to
Round-Robin	Alternates packets across subflows	Homogeneous paths; underperforms with heterogeneous RTT/loss [LIT: 5]
minRTT (kernel default)	Prefers the lowest-RTT subflow with congestion-window room	General purpose; can misjudge under RTT estimation noise, favouring a loaded fast path over an idle slow one and causing head-of-line (HoL) blocking [LIT: 5]
BLEST	Estimates and actively avoids receiver-side HoL blocking before using a slow subflow [LIT: 5]	Heterogeneous paths (e.g. Wi-Fi and cellular)

Scheduler	Behaviour	Best suited to
ECF (Earliest Completion First)	Estimates each subflow’s packet arrival time and schedules for earliest completion; behaves like minRTT/BLEST when both paths have spare window [LIT: 6]	Heterogeneous paths, latency-sensitive traffic
Redundant	Duplicates packets across all subflows [LIT: 5]	Loss-critical, low-throughput traffic (e.g. VoIP signalling)

Reasoning from these mechanisms, not from a specific study: file transfer should benefit most from throughput-maximising scheduling (Round-Robin/minRTT on symmetric paths), while interactive video and VoIP should benefit more from HoL-aware scheduling (BLEST/ECF), since one blocked subflow can stall an entire frame. VoIP’s small packets and tight jitter budget make Redundant scheduling attractive on unreliable paths, at the cost of doubled bandwidth use. This paragraph is our synthesis, not a cited result.

2.4 Kernel Architecture, History, and Design Philosophy

The Linux MPTCP implementation is built around a **meta-socket** that reorders incoming data and schedules outgoing data across subflows, with a **path manager** creating and removing subflows and a modular **scheduler** deciding how traffic is spread across them; this terminology and division come from the original implementation draft and are preserved in the current Linux kernel networking documentation [LIT: 7]. The Linux implementation began as an out-of-tree patch started around 2009 by Sebastien Barre, and the first pieces landed in the mainline Linux kernel with version 5.6 in March 2020, roughly eleven years later [LIT: 7].

MPTCP’s guiding design constraint, stated directly in RFC 8684, is that the protocol must function through the vast majority of existing middleboxes and so must resemble ordinary TCP on the wire as far as possible [LIT: 1]. This is why MPTCP is carried in TCP options rather than a new protocol.

2.5 The Middlebox Problem and Theory-versus-Practice Gaps

The same TCP-options design is also MPTCP’s biggest practical constraint. The TCP header’s Data Offset field is 4 bits, capping the header at 60 bytes total; with the fixed 20-byte header this leaves a maximum of 40 bytes for all options combined, shared with other options like timestamps and SACK [LIT: 1]. Middleboxes are also unpredictable in practice, and RFC 8684 documents that if a middlebox strips MPTCP’s options, the connection falls back to regular single-path TCP rather than failing outright [LIT: 1]. Raiciu et al.’s deployment study found a related underbuffering effect: on some topologies, one slow subflow could drag the whole connection’s throughput down; their proposed buffer-management and reordering fixes recovered up to a tenfold throughput improvement in that case, matching what a single good interface would achieve on its own [LIT: 8].

2.6 QUIC and MP-QUIC

QUIC runs over UDP as a userspace transport and integrates its cryptographic and transport handshakes; it is an encrypted transport, which limits the ability of middleboxes to inspect or modify it the way they do with plain TCP [LIT: 9]. It multiplexes independent streams so loss on one stream does not block the others, the classic HoL-blocking problem that affected TCP-based HTTP/2 [LIT: 9]. QUIC also uses unique packet numbers to avoid the retransmission ambiguity that TCP’s sequence numbers can create [LIT: 9].

MP-QUIC builds on this by letting one QUIC connection use several paths at once, not just migrate between them. Two mechanisms are central:

- **Path validation:** before using a new path, an endpoint sends a `PATH_CHALLENGE` and requires a matching `PATH_RESPONSE`, confirming the path is live before committing data to it.
- **The packet-number-space debate:** QUIC packets are numbered within a “packet number space,” and the open design question for MP-QUIC was whether to keep one shared space across all paths or give each path its own. De Coninck’s NS-3 study of both options found that per-path numbering makes the connection considerably more robust to how a specific receiver happens to generate acknowledgements,

whereas a shared space’s effective throughput depends more heavily on that receiver behaviour [LIT: 11].

Why MP-QUIC exists at all: the extend-versus-build-fresh debate. MPTCP had to work around what TCP could not be changed to do, because TCP is ossified into kernels and middleboxes worldwide; Langley et al. describe this same ossification problem as the reason QUIC was built fresh over UDP in userspace instead of extending TCP [LIT: 9]. This is also why MP-QUIC’s path validation and per-path packet numbering look different from MP_JOIN/MP_CAPABLE: QUIC does not need to smuggle its signalling past middleboxes the way MPTCP does, since almost all of a QUIC packet is already encrypted [LIT: 9].

3. Architecture: Our Testbed

Our MPTCP experiments used a two-path topology between a sender and receiver, with per-path bandwidth, delay, and loss shaped independently. Tooling, as specified in the project brief:

Layer	Tool
Multipath transport	Linux kernel MPTCP
Path/subflow control	Multipath-enabled <code>iproute2</code>
Network impairment	NetEm (bandwidth, delay, loss shaping per path)
Traffic generation and measurement	<code>iperf3</code>
Packet-level verification	Wireshark

Each of the four experiments (Section 5.1) reused this same two-path rig with different NetEm parameters. One limitation for the record: we retained `iperf3`’s summary throughput/RTT/jitter output for each run, but not raw `iperf3` logs or packet captures, so the numbers below are summary measurements rather than fully re-analysable traces.

On scheduler configuration: the exact MPTCP scheduler and path-manager mode (signal vs. fullmesh) used for these runs were not explicitly pinned down during setup. We therefore describe the observed traffic-distribution behaviour rather than asserting it was produced by a specific named scheduler.

4. MP-QUIC’s Design, Mirrored to Our Scenarios

We did not build or run an MP-QUIC testbed. For each of our four MPTCP scenarios we reasoned qualitatively about how a documented MP-QUIC implementation would likely behave, using the mechanisms in Section 2.6 and the measured trends reported by De Coninck and Bonaventure for comparable topologies [LIT: 10]. These are marked [PREDICTED] throughout Section 5.2 and are explicitly not our own experimental results.

5. Implementation and Evaluation

5.1 MPTCP: Measured Results [MEASURED]

Table 1. Testbed parameters per experiment

Exp.	Path 1 (BW / Delay / Loss)	Path 2 (BW / Delay / Loss)	Duration	Scenario
E1	50 Mbps / 10 ms / 0%	50 Mbps / 10 ms / 0%	30 s	Symmetric paths
E2	50 Mbps / 10 ms / 0%	5 Mbps / 80 ms / 0%	30 s	Asymmetric bandwidth and delay
E3	50 Mbps / 10 ms / 0%	10 Mbps / 40 ms / 3%	30 s	Asymmetric paths, Path 2 lossy
E4	50 Mbps / 10 ms / 0%	50 Mbps / 10 ms / 0%, fails mid-run	60 s	Path failure / failover

Table 2. Measured outcomes

Exp.	Throughput (send/recv)	Path 1/2 util.	RTT P1/P2	Retrans.	Surv.	Recovery
E1	95.4 / 93.6 Mbps	50.0% / 50.0%	20.65 / 20.83 ms	0	Yes	N/A
E2	52.5 / 51.2 Mbps	90.9% / 9.1%	20.82 / 161.24 ms	0	Yes	N/A
E3	42.2 / 41.6 Mbps	97.7% / 2.3%	20.78 / 80.99 ms	0 (iperf3)	Yes	N/A
E4	79.9 / 78.5 Mbps (avg.)	not measured per-path	not measured	0	Yes	approx. 1-3 s

Expected vs. actual, per experiment:

- **E1 (symmetric):** expected roughly equal traffic split and combined throughput near the sum of both paths. Actual: 50.04%/49.96% split, 95.4/93.6 Mbps aggregate, zero retransmissions; matches expectation, since with identical bandwidth, delay, and loss on both paths, MPTCP has no reason to favour either.
- **E2 (asymmetric):** expected the faster, lower-delay Path 1 to dominate. Actual: 90.94%/9.06% split, matching expectation given the 10x bandwidth and 8x delay gap between the paths.
- **E3 (asymmetric plus loss):** expected the connection to survive with Path 2 contributing even less than in E2 once loss is added. Actual: split moved to 97.67%/2.33% (more skewed than E2), connection survived, and iperf3 reported zero retransmissions at the measured layer; matches the qualitative expectation that loss makes Path 2 less attractive, though the retransmission count itself was lower than expected going in.
- **E4 (failure):** expected the connection to survive Path 2’s failure with a brief throughput dip while the failure is detected and traffic moves to Path 1. Actual: throughput dropped to 0 Mbps for about 1 second, then recovered to roughly 45 Mbps on the surviving path, connection intact throughout; matches expectation, and is consistent with MPTCP relying on retransmission-timeout-based failure detection rather than an active path-liveness probe.

All four experiments matched their pre-stated expectations; there were no surprises significant enough to warrant a design change during the two weeks.

5.2 MP-QUIC: Design-Mirrored Reasoning [PREDICTED]

Cross-checked against the measured trends in [LIT: 10]:

- **E1 analogue (symmetric):** expect a similarly even split across both paths; MP-QUIC’s per-path packet-number spaces should not meaningfully change behaviour when both paths are identical, since there is no RTT-estimation ambiguity to exploit either way.
- **E2 analogue (asymmetric BW/delay):** MPTCP’s Linux scheduler can, in some cases, keep bursting onto the fast path even after it is already loaded, because of RTT-sampling ambiguity; MP-QUIC’s cleaner per-path RTT accounting is reported to reduce this specific effect [LIT: 10]. We would predict a similarly fast-path-dominant split, but with less of the burst-then-stall behaviour MPTCP is prone to under asymmetry.
- **E3 analogue (asymmetric plus loss):** MP-QUIC’s per-stream loss recovery, with no TCP-style single sequence-number space to reorder, should isolate the lossy path’s retransmissions more cleanly. We would predict Path 1 dominance similar to E3’s actual result, but with fewer transfer-level stalls attributable to the lossy Path 2.
- **E4 analogue (failure):** MP-QUIC detects a dead path via failed `PATH_CHALLENGE/PATH_RESPONSE` probing rather than TCP’s retransmission-timeout heuristic. Probing-based detection is not automatically faster; it depends on the configured probe interval, so we do not assume MP-QUIC recovers faster than MPTCP’s approximately 1 to 3 seconds observed here. If the probe interval is set conservatively, to avoid false positives on jittery links, MP-QUIC’s failover could plausibly take longer than MPTCP’s RTO-based detection, not shorter. We flag this as a genuinely open question rather than assume the newer protocol automatically wins.

Independently of our four scenarios, the closest available measured comparison is De Coninck and Bonaventure’s evaluation, using their MP-QUIC implementation (an extension of `quic-go`) and OLIA congestion control on both protocols for a fair comparison, since no multipath variant of CUBIC exists [LIT: 10]:

- In a loss-free single-stream 20 MB download, their MP-QUIC implementation was about 13% faster than their MPTCP setup [LIT: 10].
- Across a wider sweep of no-loss, large-file download scenarios with varied per-path bandwidth and delay, MP-QUIC outperformed MPTCP in roughly 85% of cases [LIT: 10].
- The stated reason is that MPTCP experiences head-of-line blocking more often than MP-QUIC in their tests, tied to ambiguity in Linux’s TCP RTT estimation causing the scheduler to keep favouring an already-loaded fast path [LIT: 10], the same mechanism reasoned about for the E2 analogue above.

6. Comparative Analysis: TCP vs MPTCP vs QUIC vs MP-QUIC

Aspect	TCP	MPTCP	QUIC (single-path)	MP-QUIC
Paths used	One	Multiple, coupled congestion control (LIA/OLIA/BALIA) [LIT: 1,2,3,4]	One	Multiple, per-path packet numbers [LIT: 10]
HoL blocking	Present (one byte stream)	Reduced, but Linux scheduler can reintroduce it under path asymmetry [LIT: 10]	Removed at stream level by design [LIT: 9]	Removed at stream level; also avoids the RTT-ambiguity source of HoL seen in MPTCP [LIT: 10]
Failure/migration	Connection breaks if the path fails	Survives failure of one subflow; RTO-based detection, approx. 1 to 3 s recovery observed in our test [MEASURED]	Can migrate across a change of address (Connection ID based) [LIT: 9]	Survives failure of one path; <code>PATH_CHALLENGE/PATH_RESPONSE</code> probing, speed depends on probe interval [PREDICTED]
Bandwidth aggregation	None	Yes; strongly favours the better path under asymmetry (E2, E3, our data) [MEASURED]	None	Yes; about 13% faster than MPTCP in a loss-free single-stream download, better in about 85% of tested asymmetric no-loss scenarios [LIT: 10]
Fairness to single-path TCP	N/A (is single-path TCP)	Explicit design goal via coupled CC; LIA can still be unfriendly on shared bottlenecks, motivating OLIA/BALIA [LIT: 2,3,4]	N/A	Same coupled-CC fairness concern applies if MP-QUIC reuses MPTCP-style coupling; De Coninck and Bonaventure used OLIA for both sides of their comparison [LIT: 10]
Middlebox compatibility	Universal	Relies on the 40-byte TCP option space surviving middleboxes, but falls back to plain TCP if MPTCP options are stripped [LIT: 1]	Encrypted and UDP-based, less middlebox interference than raw TCP options [LIT: 9]	Same UDP/encryption advantage as QUIC [LIT: 9]
Deployment maturity	Universal	Mainlined in Linux 5.6 (2020), about eleven years after the first out-of-tree patch [LIT: 7]	Widely deployed [LIT: 9]	Research prototypes; the packet-number-space design itself was still unsettled as of 2021 to 2022 [LIT: 11]

The recurring theme, visible in both our own numbers and the literature, is that MPTCP’s biggest practical weakness (a kernel scheduler whose RTT estimate can get confused under asymmetric load) is the class of problem MP-QUIC’s per-path design was built to avoid. Neither protocol removes the more fundamental trade-off: bandwidth aggregation makes a connection more complex to reason about and, in both designs, still needs a coupled congestion-control scheme to stay fair to ordinary single-path traffic.

6.1 Inter-Library Note: TQUIC vs. Cloudflare quiche

Scoped deliberately to a design-level comparison, not a timing benchmark: no published multipath timing comparison exists between the two libraries, since `quiche`’s multipath support was still in development at the time of TQUIC’s own published comparison [LIT: 12].

	TQUIC (Tencent)	quiche (Cloudflare)
Language	Rust	Rust
Approx. size	about 50,000 LOC	about 40,000 LOC
Multipath QUIC	Supported	In development (not shipped) at time of comparison [LIT: 12]
Primary deployment	Tencent Cloud EdgeOne	Cloudflare edge network
Congestion control options	CUBIC, BBR, BBRv3, COPA	CUBIC, BBR (varies by version)

TQUIC’s own published benchmark reports it outperforming several other QUIC implementations, including `quiche`, on general request-handling throughput under load [LIT: 12], but that benchmark measures single-path HTTP/3 request throughput, not multipath behaviour, and it is a vendor-published result rather than an independently reproduced one, so we treat it as directional context rather than a number we can defend precisely. The comparison we can make with confidence is architectural: at the time of writing, TQUIC is the only one of the two with multipath support to actually evaluate, which is why our multipath discussion above relies on De Coninck and Bonaventure’s `quic-go`-based implementation rather than either of these two libraries specifically.

7. Non-Functional Testing Parameters

- **Performance.** Aggregate throughput scaled with the number of usable paths in our MPTCP data: a near-linear combination on symmetric paths (E1: about 95 Mbps against a 100 Mbps combined ceiling) [MEASURED], degrading gracefully rather than linearly once asymmetry or loss is introduced (E2, E3) [MEASURED].
- **Scalability.** Neither protocol was tested beyond two subflows here. RFC 8684 lets a connection cap how many subflows or additional addresses it accepts, via `subflows` and `add_addr_accepted` [LIT: 1]. We did not measure behaviour beyond two paths, so this remains untested rather than a claimed result.
- **Reliability.** Both protocols are designed to survive a single-path failure. We measured MPTCP’s actual failover (approx. 1 to 3 s, throughput dip to 0 Mbps for about 1 s) [MEASURED]; MP-QUIC’s equivalent is [PREDICTED] only, and we specifically flag that its probe-based detection is not guaranteed to be faster (Section 5.2).
- **Fairness.** Coupled congestion control is what keeps either protocol from starving ordinary single-path traffic on a shared bottleneck; LIA’s known unfriendliness under some conditions [LIT: 3] is the direct motivation for OLIA and BALIA, and for De Coninck and Bonaventure’s choice to standardise on OLIA for both sides of their MPTCP/MP-QUIC comparison [LIT: 10].
- **Compatibility.** MPTCP falls back to plain TCP if a middlebox strips its options, losing multipath behaviour but not the connection [LIT: 1]; QUIC/MP-QUIC’s UDP-and-encrypted design sidesteps TCP-option-stripping, though UDP itself is occasionally blocked or rate-limited on some networks, a different compatibility risk rather than a strictly smaller one.

8. Recommendations

The following is our own synthesis of the mechanisms and results above, not additional cited findings. There is no single best protocol; the right choice depends on the application’s tolerance for latency, jitter, and loss:

- **Video streaming** favours HoL-aware scheduling (BLEST/ECF on MPTCP) or MP-QUIC’s stream-level isolation, since a stalled subflow should not stall the whole frame; our E2/E3 results show a well-tuned multipath connection can still use a slow, lossy secondary path opportunistically rather than wasting it.
- **File transfer** favours raw throughput; our E1 result (near-linear aggregation on symmetric paths) and the literature’s 13%/85% MP-QUIC-over-MPTCP figures [LIT: 10] both point toward MP-QUIC as

the stronger choice where it is deployable, with MPTCP as the more mature fallback where kernel-level multipath is already available.

- **VoIP** favours low jitter and fast loss recovery over raw throughput; Redundant scheduling on MPTCP, or MP-QUIC’s independent per-path loss recovery, both fit better than throughput-maximising schedulers, at the cost of extra bandwidth use.

9. Challenges Faced

- Configuring kernel-level MPTCP (path-manager mode, sysctls) alongside NetEm shaping and multipath-enabled `iproute2` took more trial and error than expected for a two-week window; we did not end up pinning down the exact scheduler/path-manager mode used for the final runs (Section 3).
- We did not retain raw `iperf3` logs or packet captures, only summary metrics, which limits how deeply the measured numbers can be re-analysed after the fact.
- Building a directly comparable topology across a kernel-space protocol (MPTCP) and a would-be userspace library (any MP-QUIC implementation) is nontrivial; this, combined with the timeline, is why MP-QUIC stayed at the literature-and-reasoning stage rather than being implemented.
- Scheduler and congestion-control heterogeneity across MPTCP and the various MP-QUIC research prototypes make any like-for-like comparison inherently approximate; even De Coninck and Bonaventure had to standardise both sides on OLIA specifically because no multipath CUBIC variant exists [LIT: 10]. This heterogeneity is also why the inter-library TQUIC/quiche question (Section 6.1) had to be scoped down to a design-level comparison rather than a timing one.

10. Conclusion

Against the project’s expected outcomes: we measured MPTCP’s throughput, latency, and reliability under four network conditions end to end; we analysed how it handles loss and a path failure specifically (E3, E4); we gained direct, hands-on experience configuring a kernel-level multipath protocol and its NetEm testbed; and we produced application-specific recommendations. What we did not do, run our own MP-QUIC experiments, is stated plainly rather than implied away, and the MP-QUIC side of every comparison in this report traces to either a cited source or explicitly labelled reasoning. The natural next step, if this project continued, would be implementing MP-QUIC (for example via `quic-go`’s multipath extension or TQUIC) on the identical NetEm testbed used here for E1 to E4, converting Section 5.2’s [PREDICTED] rows into real measurements directly comparable to our own MPTCP data.

11. References

1. IETF, *RFC 8684: TCP Extensions for Multipath Operation with Multiple Addresses*, 2020.
2. C. Raiciu, M. Handley, D. Wischik, *RFC 6356: Coupled Congestion Control for Multipath Transport Protocols*, IETF, 2011.
3. R. Khalili, N. Gast, M. Popovic, J.-Y. Le Boudec, “Non-Pareto Optimality of MPTCP: Performance Issues and a Possible Solution,” 2012.
4. Q. Peng, A. Walid, J. Hwang, S. H. Low, “Multipath TCP: Analysis, Design and Implementation” (the BALIA algorithm), ACM SIGMETRICS, 2013.
5. S. Ferlin, O. Alay, O. Mehani, R. Boreli, “BLEST: Blocking Estimation-based MPTCP Scheduler for Heterogeneous Networks,” IFIP Networking, 2016.
6. Y.-S. Lim et al., “ECF: An MPTCP Path Scheduler to Manage Heterogeneous Paths,” 2017.
7. Linux Kernel networking documentation, “Multipath TCP (MPTCP),” docs.kernel.org; and the MultiPath TCP - Linux Kernel Implementation project, multipath-tcp.org (implementation history and architecture terminology).
8. C. Raiciu, C. Paasch, S. Barre, A. Ford, M. Honda, F. Duchene, O. Bonaventure, M. Handley, “How Hard Can It Be? Designing and Implementing a Deployable Multipath TCP,” USENIX NSDI, 2012.
9. A. Langley et al., “The QUIC Transport Protocol: Design and Internet-Scale Deployment,” ACM SIGCOMM, 2017.
10. Q. De Coninck, O. Bonaventure, “Multipath QUIC: Design and Evaluation,” ACM CoNEXT, 2017.
11. Q. De Coninck, “The Packet Number Space Debate in Multipath QUIC,” 2021-2022.
12. TQUIC Project (Tencent), “Comparison to Alternatives,” official documentation, tquic.net.